



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming
Assignment 11: Advanced synchronization mechanisms
Spring Semester 2026

Overview

In this exercise, we take a closer look on how to implement efficient parallel data-structures. We simulate a common scenario in data processing, a parallel list. Based on your knowledge about locks, you will implement the queue using different lock strategies, (e.g., coarse and fine grained locking, optimistic locking, lazy locking) and measure the performance of your implementation.

Getting Prepared

- Fetch the new assignment from Gitlab by running:

```
git fetch upstream  
git merge upstream/main
```

You should now see the new assignment appear in your local directory.

- Open the project in IntelliJ IDEA: Click on *File* in the top-menu, then select *Open*. Select the folder of the new assignment.

Look at the two source files `SortedListInterface` and `SequentialList` in the `assignment10` package. The `SequentialList` is a simple and non thread-safe implementation of `SortedListInterface` which specifies an API for a list that keeps its elements always in sorted order. The tests in `BenchmarkTest` simulate a producer/consumer scenario with different threads that produce items (by adding them to the list) and consume items (by removing them from the list) as well as doing many read-only accesses to the list by checking if an item is in the list.

Note that for the purpose of this exercise you are not supposed to rely on existing `List`, `Map` implementations from the Java standard library for your implementation, but rather you are supposed to implement your own. However, you are of course allowed to study and re-use ideas from the source code of those implementations.

Your task is to implement four thread-safe versions of `SortedListInterface` using different lock strategies:

Coarse Grained Locking

The first implementation is supposed to be a simple and thread-safe version of the `SortedListInterface`, using a single, global lock to make sure update operations do not interfere with each other. Implement the provided template class `CoarseGrainedLockList`.

Coarse grained locking method locks the whole list when a thread requires to *add*, *remove* or check if the list *contains* the method. This is the simplest way of implementing the thread safe linked list, but gives the worst performance.

Fine Grained Locking

As you know, having a big global lock that locks the whole data-structure is simple to implement, but usually tends to hurt performance. In your next implementation of `SortedListInterface`, you will address this by using individual locks for each queue element.

Implement the class `FineGrainedLockList` that uses an individual lock per list element. For your implementation, you should be careful to lock any nodes that you need in order to update the list, but not too many! You need to be sure that between comparison of the list values and changing the list content no other threads can access the data. Therefore, hand-over-hand locking is a good choice.

Optimistic Locking

One way to reduce synchronization costs further is to take a chance: search without acquiring locks, lock the nodes found, and then confirm that the locked nodes are correct. If a synchronization conflict causes the wrong nodes to be locked, then release the locks and start over. Normally, this kind of conflict is rare, which is why we call this technique optimistic synchronization.

Implement the class `OptimisticLockList` that still uses an individual lock per list element. This method typically requires a re-validation if the data-structure has been modified after the lock was acquired. Think about what could go wrong here. *Hint: What can happen between value comparison and locking of the nodes? How can you check if the data was modified?*

(Optional) Lazy Locking

This exercise is optional because it is unlikely that Lazy Locking was already discussed in the lecture when the exercise is supposed to be solved.

The `OptimisticLockList` implementation works best if the cost of traversing the list twice without locking is significantly less than the cost of traversing the list once with locking. One drawback of this particular algorithm is that `contains()` acquires locks (why?), which is unattractive if `contains()` calls are much more common than calls to other methods.

Implement `LazyLockList` so that `contains()` calls are wait-free, and `add()` and `remove()` methods, while still blocking, traverse the list only once (in the absence of contention). The trick to implementing lazy locking is to keep track of deleted nodes by adding a Boolean `marked` field to each node. We maintain the invariant that every unmarked node is reachable which allows us to avoid re-traversing the whole list. If a node is marked, then that node is considered not reachable and therefore considered as not contained in the list. To remove an element, we take two steps: First, logically remove the target node by marking it as removed, and second, physically remove it by redirecting the reference in the predecessor's next field.

Implementation performance

Once you are confident that your implementations are correct, run the `Main` method of the `BenchmarkTest` class, which will benchmark the amount of operations per second for each of your implementations by run-

ning a pre-defined workload. In your report, list the numbers you achieve for each of the implementations and briefly discuss your results.

You can also play with the initial parameters `numOps`, `initialElems`, `insertPercentage` in `BenchmarkTest` to see if they impact your results.

It is very difficult to write a JUnit test which detects all possible race conditions with a high probability. `SortedLinkedListTest` tries to detect some errors by adding and removing elements to and from the list with many threads in parallel. You can use the test to find bugs in your implementation but you should not rely on it to find all errors. Feel free to adjust the parameters of the test or modify it if you have a good idea on how to test your implementation.

Submission

You need to submit your code and reports by using our submission system. This step will be the same for all of the subsequent exercises, but we will explain it in detail here so you can familiarize yourself with the system.

To submit your code using the terminal, follow the instructions below:

- In the terminal, navigate to your personal repository.
- Add your changes:

```
git add . # this command adds all changes in the git repo
```
- Commit your changes. In your commit message, make sure to include the [Submission] tag.

```
git commit -m "[Submission] ..."
```
- Push your changes!

```
git push
```

To submit your code using IntelliJ, follow the instructions below:

- Press the “Commit” button (found on the left hand toolbar)
- Select all the files you want changed during the assignment.
- Write a descriptive commit message. Make sure to include the [Submission] tag.
- Press “Commit and Push”

After this step, you can check your repository on the GitLab website in your browser. Make sure that you see your changes there!